FACULTAD DE INGENIERÍA
UNIVERSIDAD DE TALCA

Tecnologías Vigentes de Bases de Datos

GESTIÓN DE BASES DE DATOS

RODRIGO PAVEZ MADARIAGA

INGENIERÍA CIVIL EN COMPUTACIÓN

TALCA UNIVERSIDAD CHILE

Introducción

La evolución de las bases de datos ha transformado el almacenamiento de información desde sistemas de archivos manuales y cintas magnéticas en los años 50 hasta las modernas bases de datos en la nube, inteligentes y NoSQL actuales.

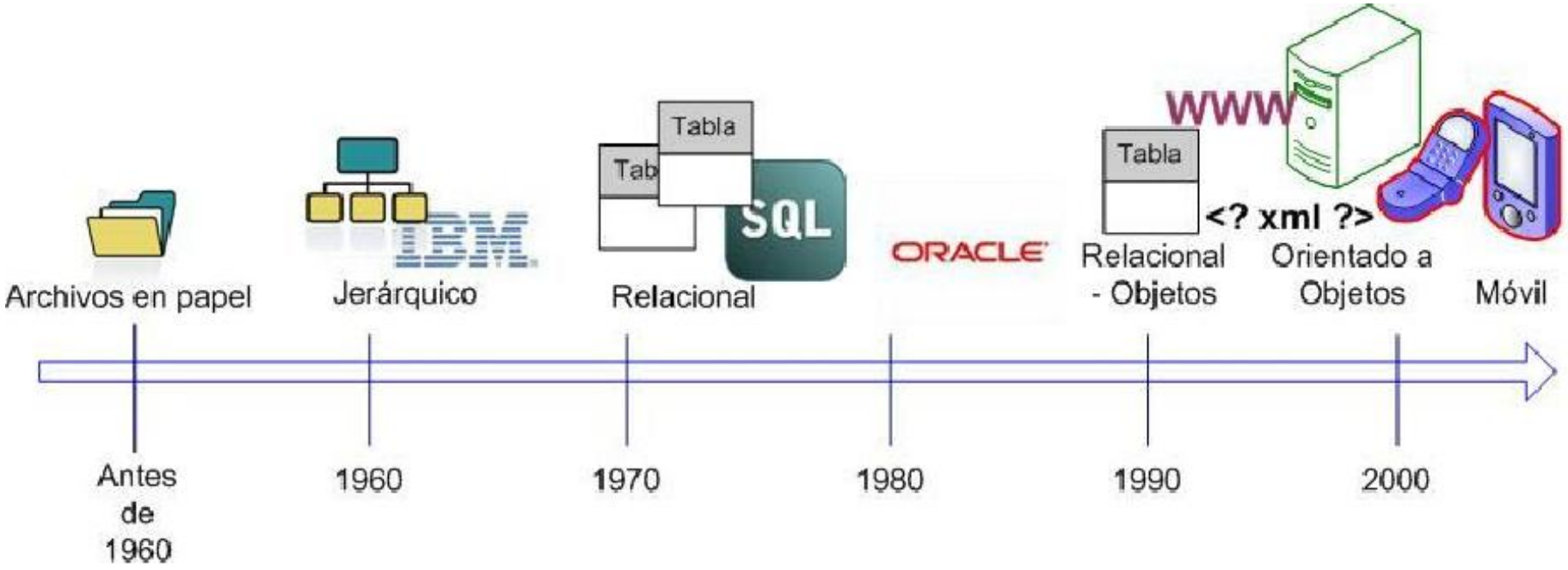
Este desarrollo se divide en etapas:

- jerárquicas/red (60s)
- relacionales SQL (70s-80s)
- orientadas a objetos (90s)
- NoSQL/Big Data (2000s-actualidad)

Buscando mejorar la eficiencia, reducir redundancia y gestionar volúmenes masivos de datos.

Introducción

Esta evolución ha permitido pasar de un procesamiento lento y manual a sistemas distribuidos, veloces y capaces de analizar grandes volúmenes de información en tiempo real para la toma de decisiones empresariales.



Bases de Datos Relacionales (SQL)

Características

- Modelo estructurado en tablas (filas y columnas)
- Uso de SQL como lenguaje estándar
- Integridad referencial mediante claves (PK/FK)
- Soporte de transacciones **ACID** (Atomicidad, Consistencia, Aislamiento, Durabilidad)

Ventajas

Alta consistencia e integridad de datos
Lenguaje estandarizado y ampliamente conocido
Ideal para sistemas transaccionales (finanzas, ERP)
Buen soporte y madurez tecnológica

Limitaciones

Escalabilidad horizontal limitada
Esquema rígido (difícil adaptación a cambios)
Menor eficiencia con datos no estructurados o masivos

Bases de Datos No Relacionales (NoSQL)

Características

- Modelo flexible o sin esquema definido
- Distribución y replicación de datos
- Tipos: documental, clave-valor, columnas, grafos
- Basadas en modelo BASE (Basically Available, Soft state, Eventually cosistent)

Ventajas

Alta escalabilidad horizontal
Flexibilidad en el manejo de datos
Alto rendimiento en grandes volúmenes
Adecuadas para Big Data y apps en tiempo real

Limitaciones

Consistencia eventual (no siempre inmediata)
Falta de estandarización
Mayor complejidad en consultas avanzadas
Dependencia del tipo de motor

Bases de Datos NewSQL

Características

- Combinan SQL tradicional con escalabili
- Soporte de transacciones ACID en entor
- Arquitectura moderna orientada al alto

Ventajas

Escalabilidad horizontal sin perder consis

Compatibilidad con SQL

Alto rendimiento en sistemas críticos

NewSQL es una categoría de sistemas de gestión de bases de datos relacionales (RDBMS) modernos que combinan la escalabilidad horizontal de NoSQL con las garantías de consistencia ACID (atomicidad, consistencia, aislamiento, durabilidad) de los sistemas SQL tradicionales. Diseñadas para cargas de trabajo OLTP de alto rendimiento, permiten gestionar grandes volúmenes de datos distribuidos sin sacrificar la fiabilidad.

Las cargas de trabajo OLTP (Procesamiento de Transacciones en Línea) se caracterizan por gestionar grandes volúmenes de transacciones pequeñas, rápidas y concurrentes, generalmente de escritura, con tiempos de respuesta inferiores a un segundo. Son fundamentales para operaciones operativas diarias como bancos, comercio electrónico, reservas y/o control de inventarios, asegurando la integridad de datos.

Google Spanner, CockroachDB, YugabyteDB, VoltDB

Comparación General

Tecnología	*Consistencia*	*Escalabilidad*	*Flexibilidad*	*Uso típico*
Relacional	Alta (ACID)	Media	Baja	Sistemas financieros
NoSQL	Media (BASE)	Alta	Alta	Big Data, apps web
NewSQL	Alta	Alta	Media	Sistemas distribuidos críticos
Cloud	Variable	Alta	Alta	Aplicaciones modernas
Grafos	Alta	Media	Media	Redes y relaciones
In-Memory	Alta	Media	Baja	Tiempo real

FACULTAD DE INGENIERÍA
UNIVERSIDAD DE TALCA

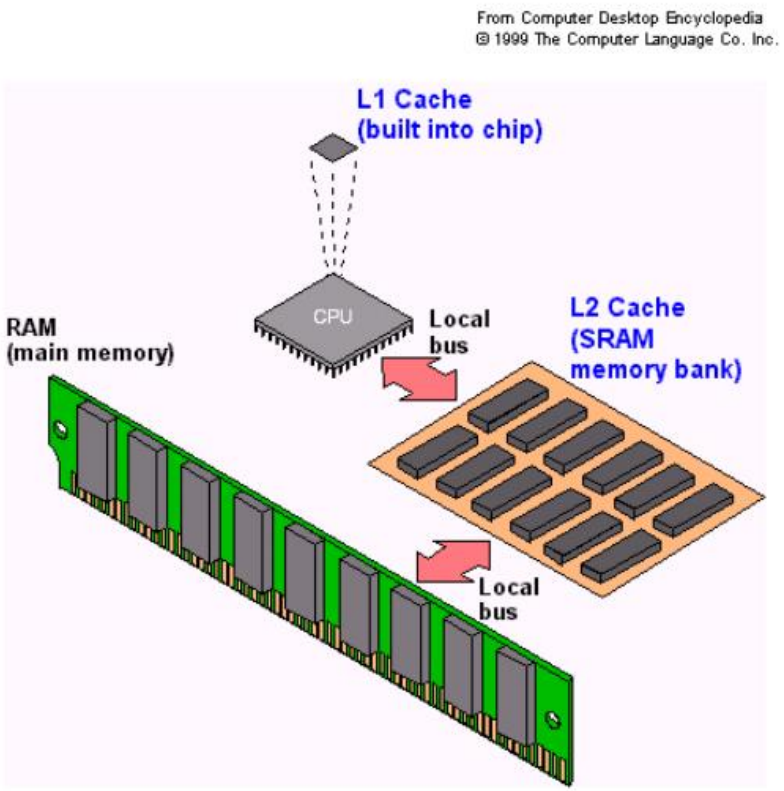
Almacenamiento e Indexación de Datos

INGENIERÍA CIVIL EN COMPUTACIÓN

TALCA UNIVERSIDAD CHILE

Jerarquías de Memorias

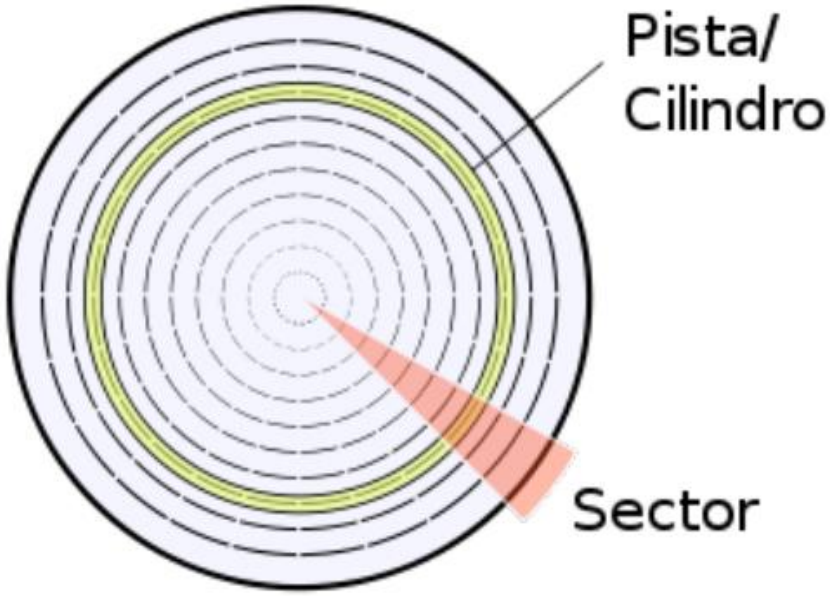
Almacenamiento principal o primario

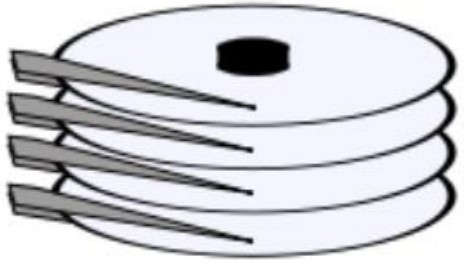


Almacenamiento secundario

Almacenamiento terciario

Almacenamiento Secundario





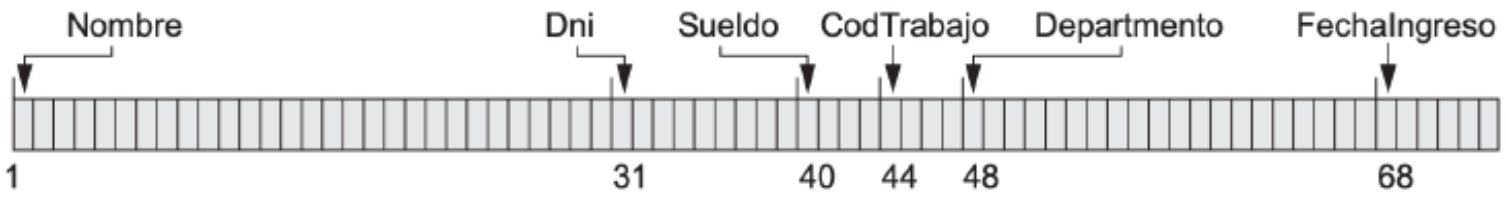
Cabezas

8 cabezas,
4 platos

Tipos de Registro

struct empleado { char nombre[30]; char dni[9]; int sueldo; int codtrabajo; char departamento[20]; }

Registros de longitud fija



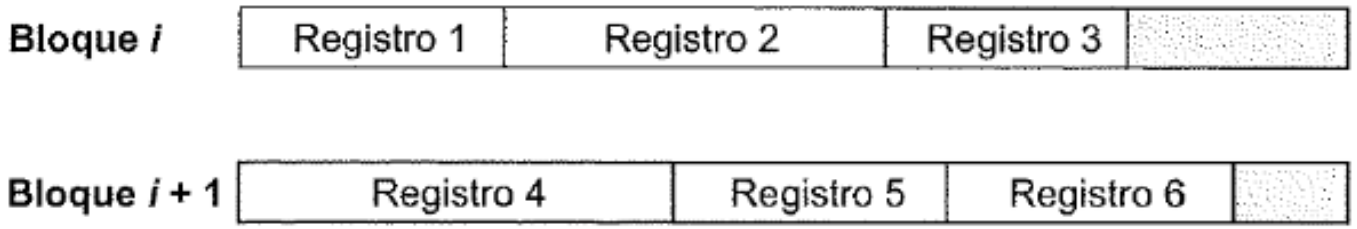
Registros de longitud variable

Caracteres separadores

- = Separa el nombre del campo del valor del campo
- ▌ Separa los campos
- ⊠ Termina el registro

Organización de Registros en Disco

Organización no extendida



Organización extendida

Operaciones sobre Ficheros

- Open
- Reset
- Find / Locate (with condition)
- Read / Get (from buffer)
- FindNext
- Delete
- Modify
- Close
- Scan
  - Find / FindNext + Read
- FindAll (cond)
- Find / Locate n
- FindOrdered
- Reorganize

Fichero Heap o de Pila

Es la forma más sencilla de organización de archivos en una base de datos, donde los registros se insertan secuencialmente en los bloques de datos sin un orden específico.

- Datos desordenados
- Inserción eficaz nuevos registros al final
- Búsqueda lineal si b = #bloques
  - caso promedio = b/2
  - peor caso = b
- Borrado: derroche de espacio

¿Alguna solución?

	Nombre	Dni	FechaNac	Trabajo	Sueldo	Sexo
Bloque 1	Wong, James					
	Wright, Pam					
	⋮					
	Acosta, Marc					
Bloque 2	Abbott, Diane					
	Zimmer, Byron					
	⋮					
	Akers, Jan					
	⋮					
Bloque *n*–1	Adams, John					
	Wood, Donald					
	⋮					
	Aaron, Ed					
Bloque *n*	Adams, Robin					
	Wyatt, Charles					
	⋮					
	Woods, Manny					

Fichero ordenado o Secuencial

Almacena registros consecutivamente, ya sea en orden de llegada o según un campo clave (ascendente/descendente)

- Datos ordenados
- Usa un campo o clave de ordenación (ej., Nombre)
- Lectura eficaz búsqueda binaria - $O(log_2 b)$
- Actualización costosa (para mantener el orden)

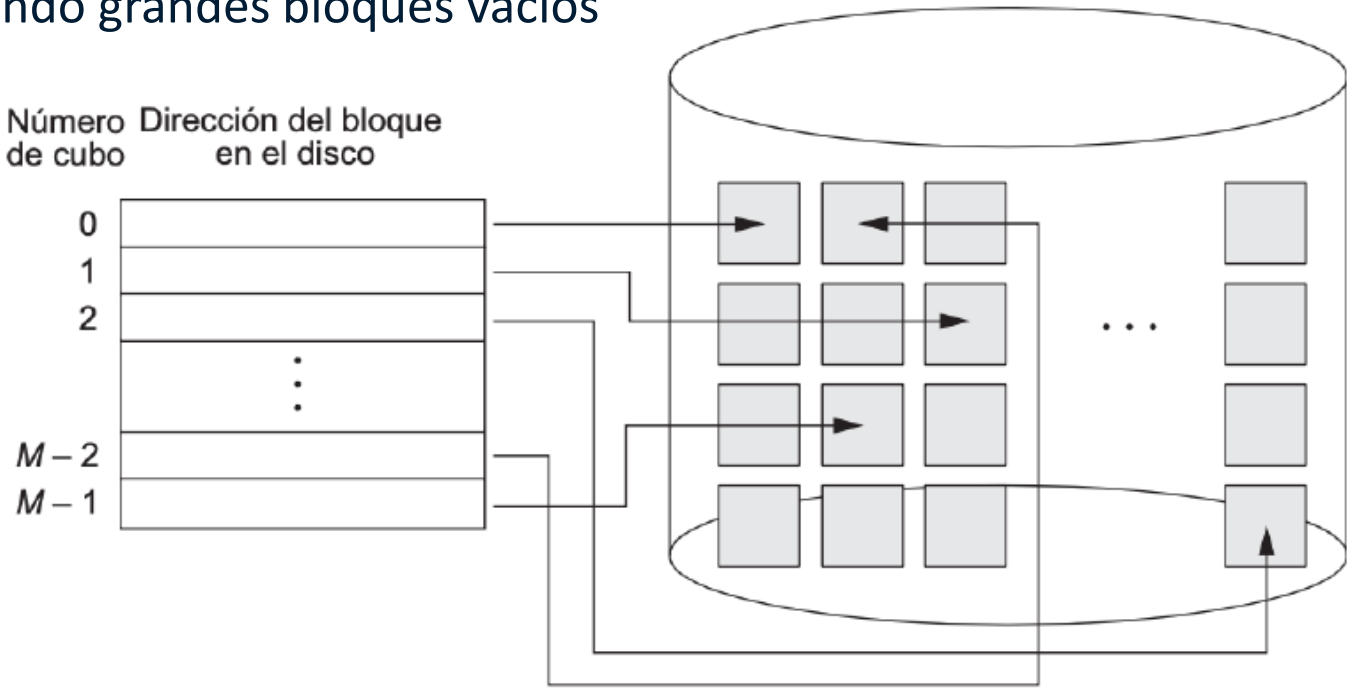
¿Alguna solución?

	Nombre	Dni	FechaNac	Trabajo	Sueldo	Sexo
Bloque 1	Aaron, Ed					
	Abbott, Diane					
	⋮					
	Acosta, Marc					
Bloque 2	Adams, John					
	Adams, Robin					
	⋮					
	Akers, Jan					
	⋮					
Bloque *n*–1	Wong, James					
	Wood, Donald					
	⋮					
	Woods, Manny					
Bloque *n*	Wright, Pam					
	Wyatt, Charles					
	⋮					
	Zimmer, Byron					

Ficheros Dispersos

Técnica de almacenamiento eficiente que permite a las bases de datos ocupar espacio en disco solo para datos reales, reemplazando grandes bloques vacíos (ceros) por metadatos

- Se basa en una función “h” de dispersión (hash)
- Un campo actúa como clave “k” de dispersión
- h(k) = dirección bloque
- Espacio de direcciones = cubos (de registros)



FACULTAD DE INGENIERÍA
UNIVERSIDAD DE TALCA

Métodos de Indexación de datos

INGENIERÍA CIVIL EN COMPUTACIÓN

TALCA UNIVERSIDAD CHILE

FACULTAD DE INGENIERÍA
UNIVERSIDAD DE TALCA

¿Qué es un índice?

■ **Parte 4** **Almacenamiento de datos, indexación, procesamiento de consultas y diseño físico 387**

Capítulo 13 **Almacenamiento en disco, estructuras básicas de ficheros y dispersión 389**

13.1 Introducción 390
13.2 Dispositivos de almacenamiento secundario 393
13.3 Almacenamiento de bloques en el búfer 398
13.4 Ubicación de los registros de fichero en disco 399
13.5 Operaciones sobre ficheros 403
13.6 Ficheros de registros desordenados (ficheros heap) 405
13.7 Ficheros de registros ordenados (ficheros ordenados) 406
13.8 Técnicas de dispersión 409
13.9 Otras organizaciones principales de ficheros 417
13.10 Paralelismo del acceso al disco mediante la tecnología RAID 418
13.11 Nuevos sistemas de almacenamiento 423
13.12 Resumen 424
Preguntas de repaso 425
Ejercicios 426
Bibliografía seleccionada 428

Capítulo 14 **Estructuras de indexación para los ficheros 429**

14.1 Tipos de índices ordenados de un nivel 430
14.2 Índices multinivel 438
14.3 Índices multinivel dinámicos utilizando árboles B y B1 442
14.4 Índices en claves múltiples 453

INGENIERÍA CIVIL EN COMPUTACIÓN

TALCA UNIVERSIDAD CHILE

Índices (indexes)

- Un índice es una estructura de datos auxiliar que acelera la recuperación de registros
- Un índice se asocia a un atributo de una tabla
- Una tabla puede tener varios índices para distintos atributos
- Tipos comunes: ficheros ordenados, árboles (B+), dispersión (hash)

Índice Principal o Primario

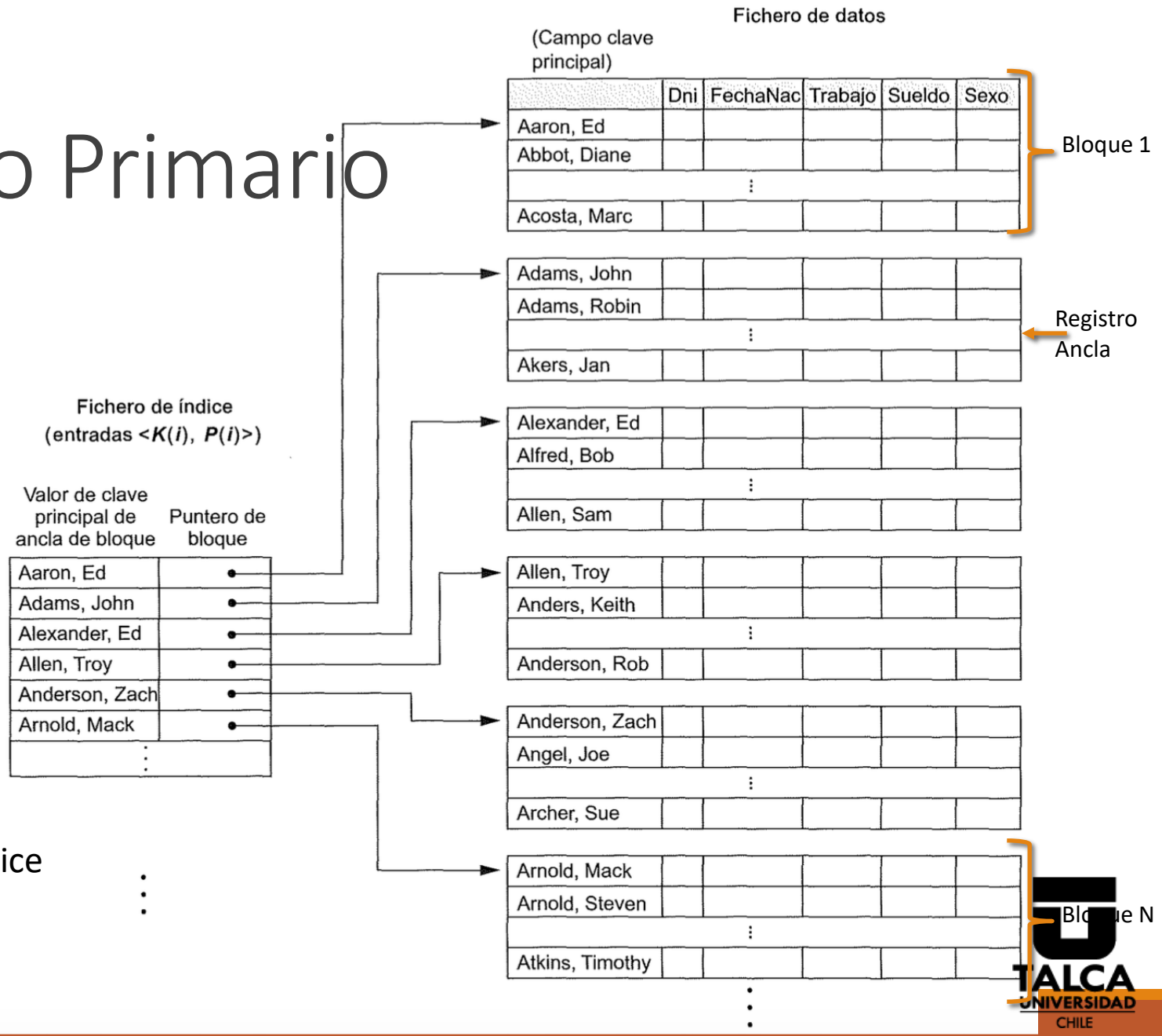
Un índice principal se especifica en el campo clave de ordenación de un fichero ordenado de registros

#entradas = #bloques

El índice es más pequeño que el fichero (índice escaso)

Búsqueda binaria

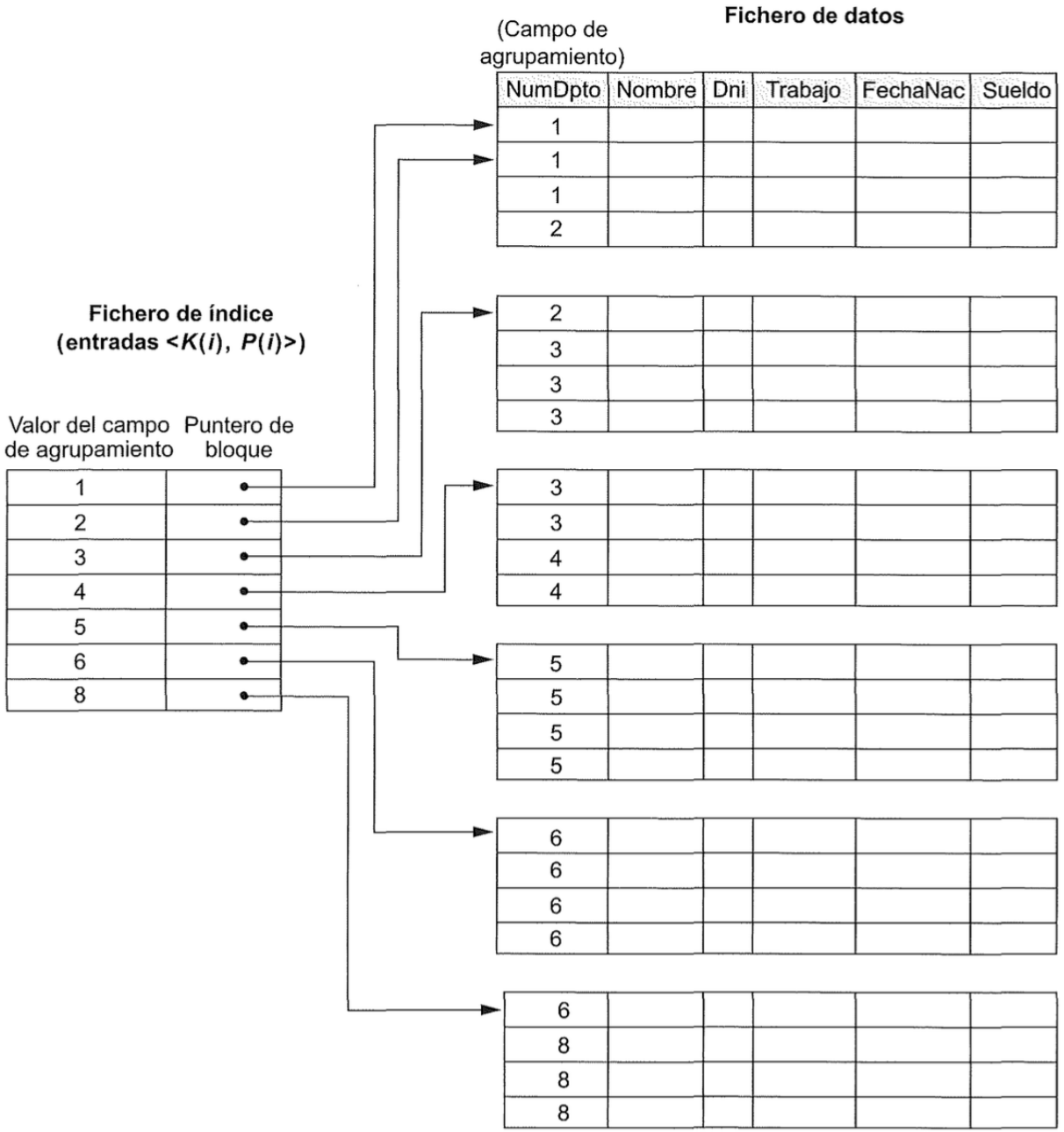
Actualización del fichero = actualización del índice



Índice Agrupado

Un índice agrupado aplica si los registros del fichero están ordenados físicamente por un **campo no clave.**

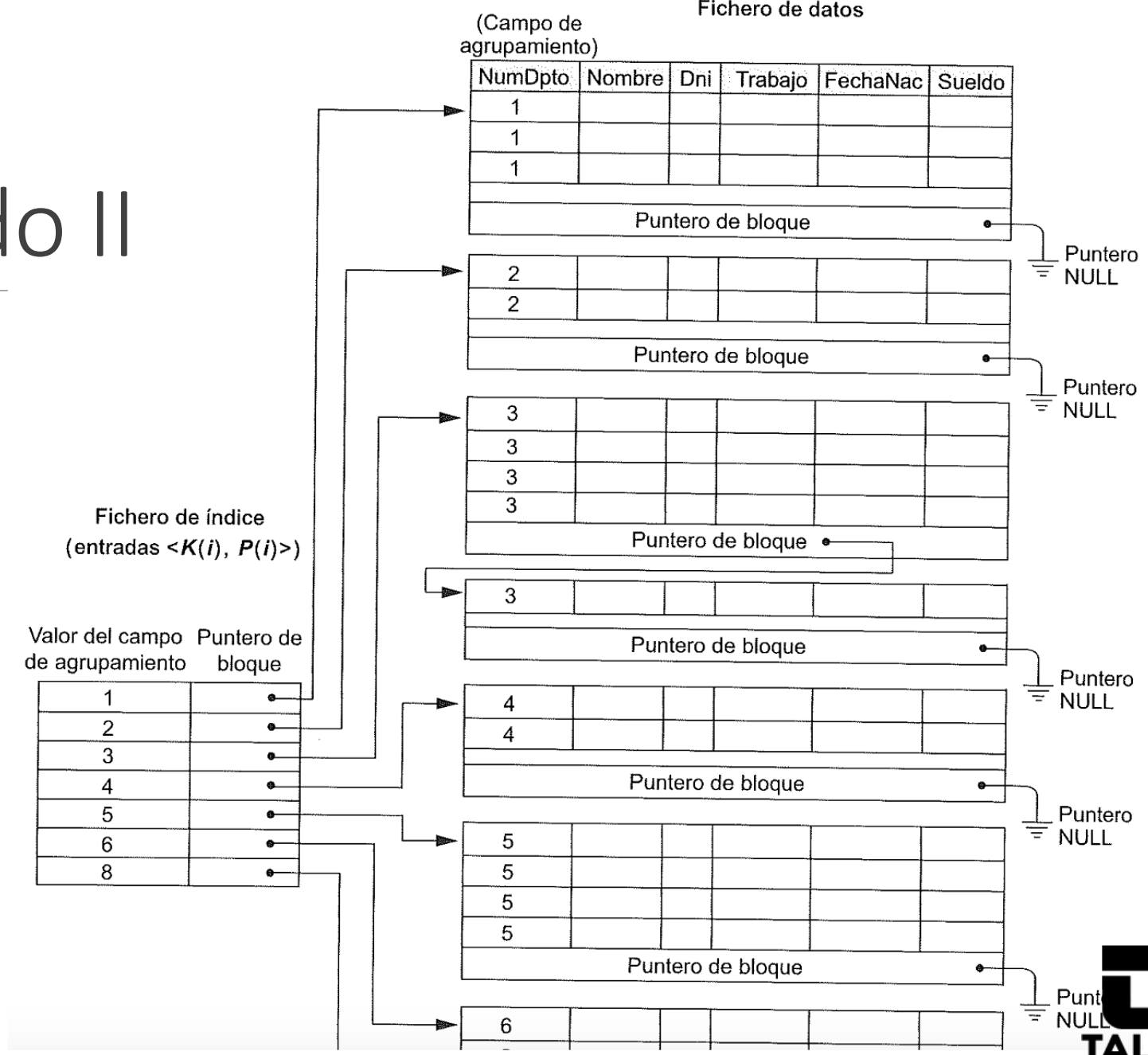
El índice tiene una entrada por cada valor distinto del campo de indexación (índice no denso)



Índice Agrupado II

El índice agrupado básico tiene problemas de inserción y borrado

Solución parcial
Reservar uno o más bloques para cada valor del campo de agrupamiento



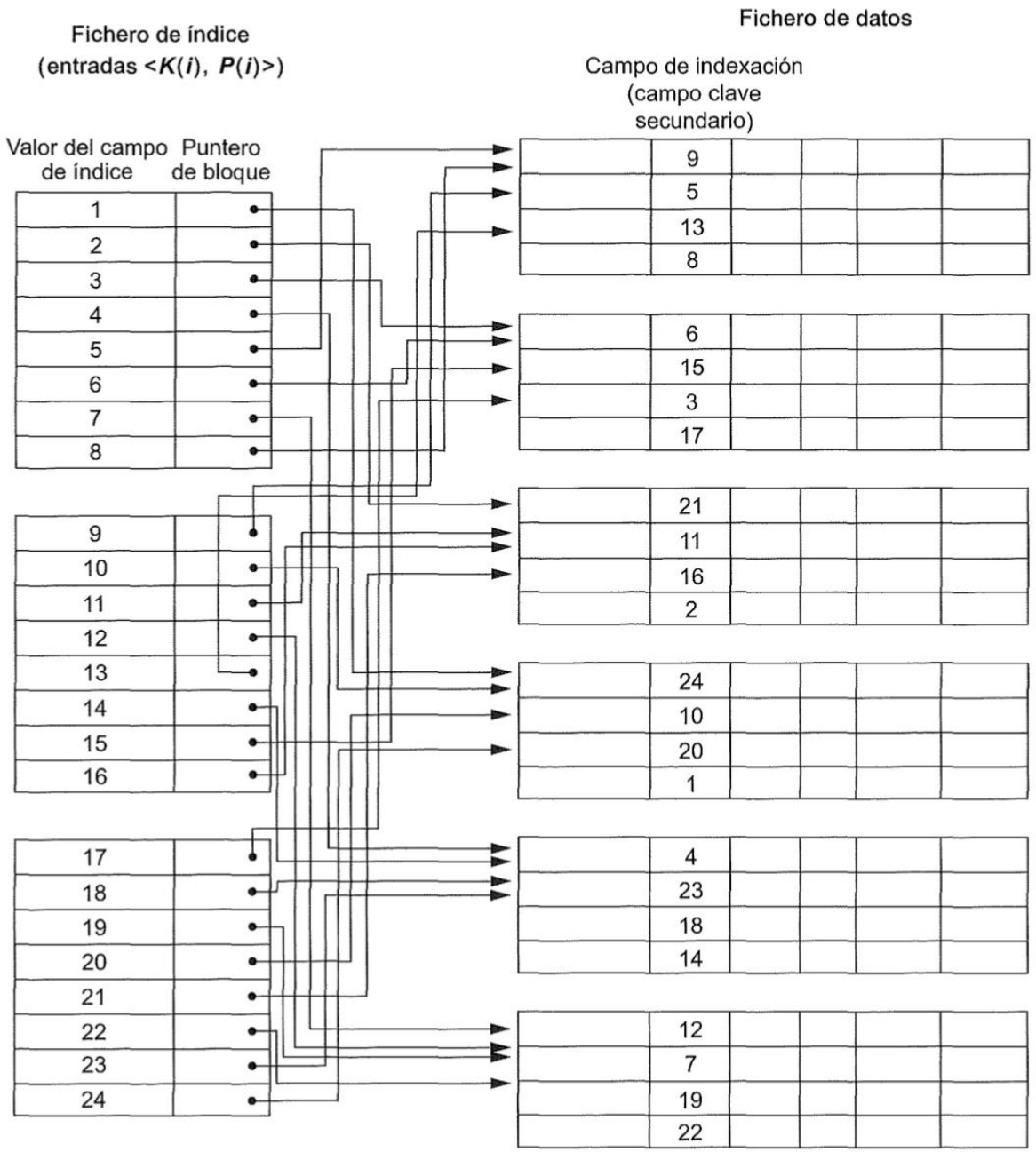
Índice Secundario

Un índice secundario es independiente del tipo de fichero de datos.

Pueden crearse para atributos de valores únicos (este ejemplo) o duplicados (siguiente ejemplo).

Permite búsqueda binaria a nivel del índice.

El índice podría ser tan grande como el fichero (índice denso)

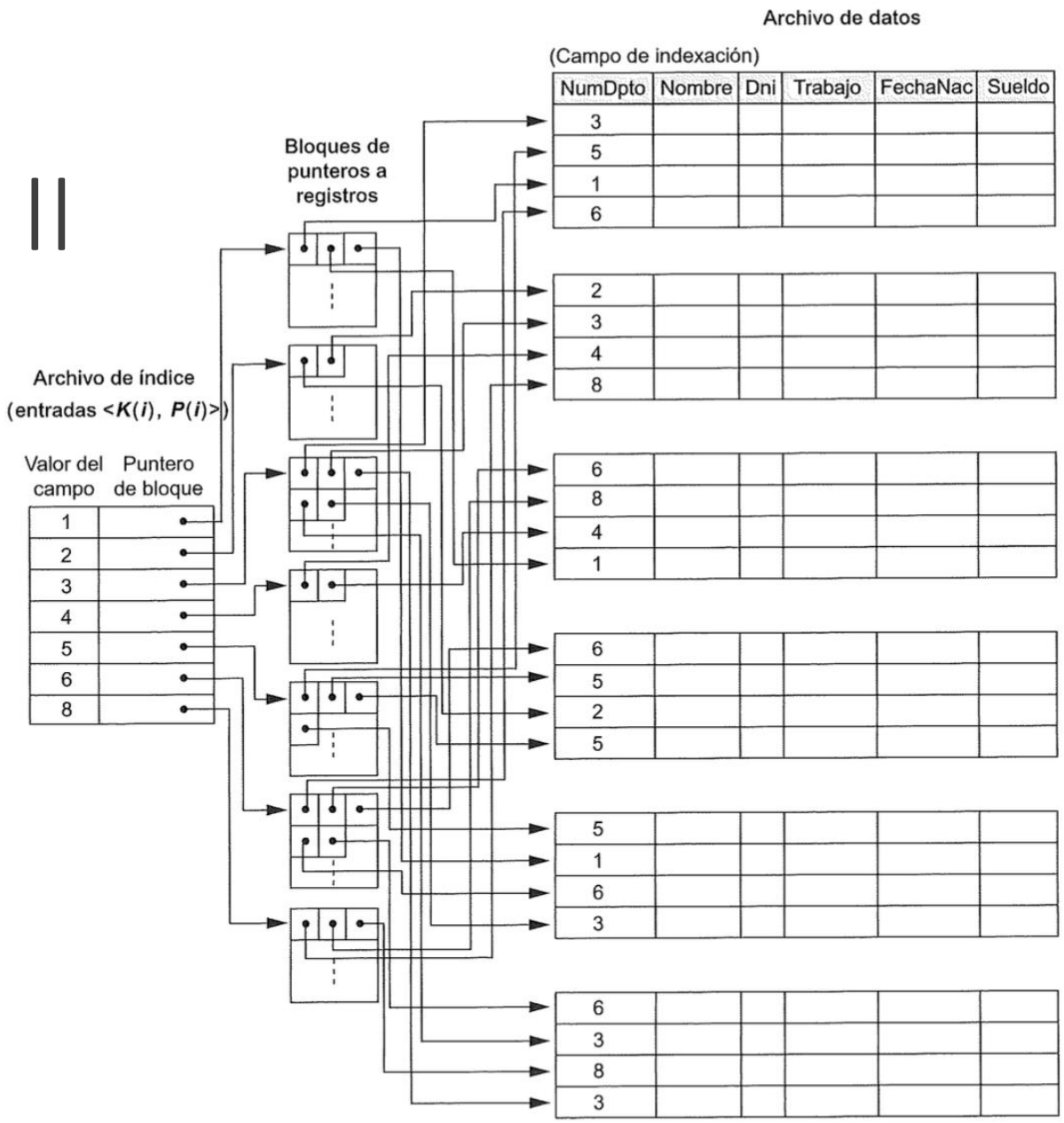


Índice Secundario II

Ejemplo de índice secundario sobre un atributo con valores duplicados.

Las entradas del índice tienen valores únicos.

Se agrega un nivel de indirección para manipular los múltiples punteros.

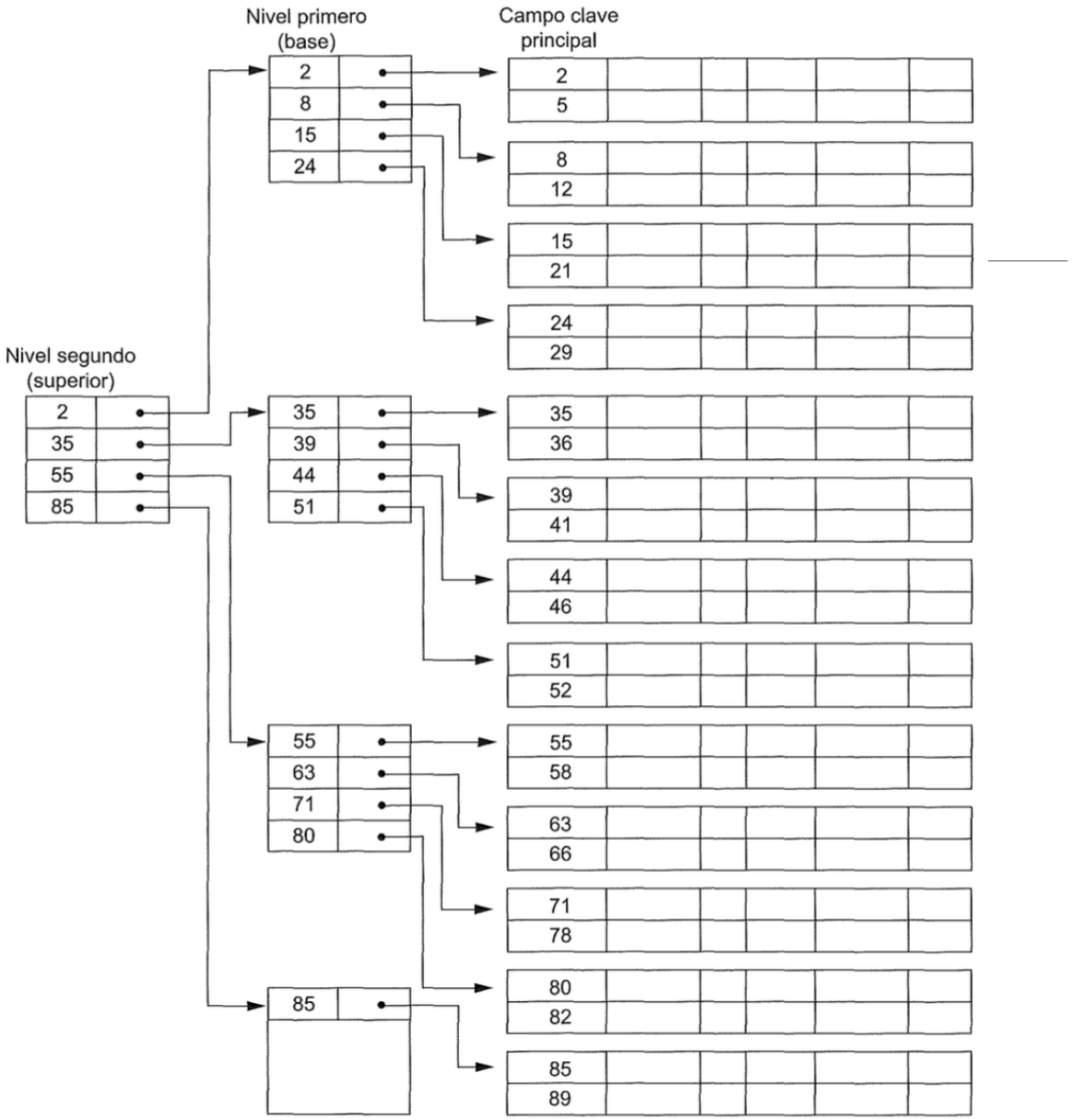


Índice Multinivel

Funciona con un fichero ordenado de registros

Ejemplo de índice de dos niveles

Permite una búsqueda más rápida



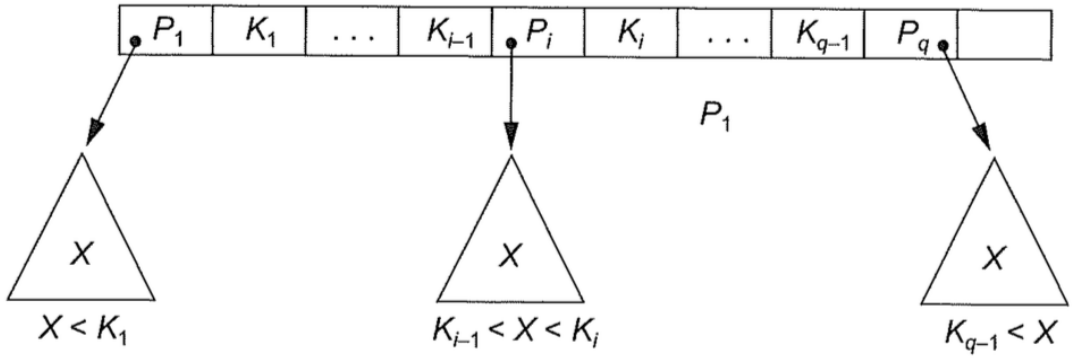
Árboles de búsqueda

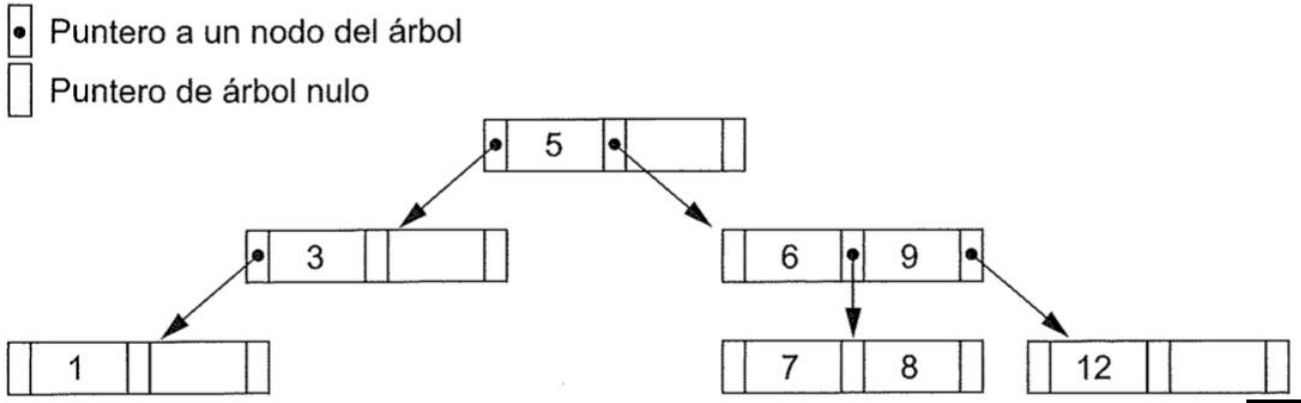
Un árbol de búsqueda permite guiar la búsqueda de un registro dado el valor de uno de sus campos.

Cada nodo de un árbol de búsqueda de orden “p” contiene, a lo sumo, “p-1” valores de búsqueda (k_i) y “p” punteros a los sub-árboles

Árbol de búsqueda de orden p = 3

Importante: mantener un árbol equilibrado



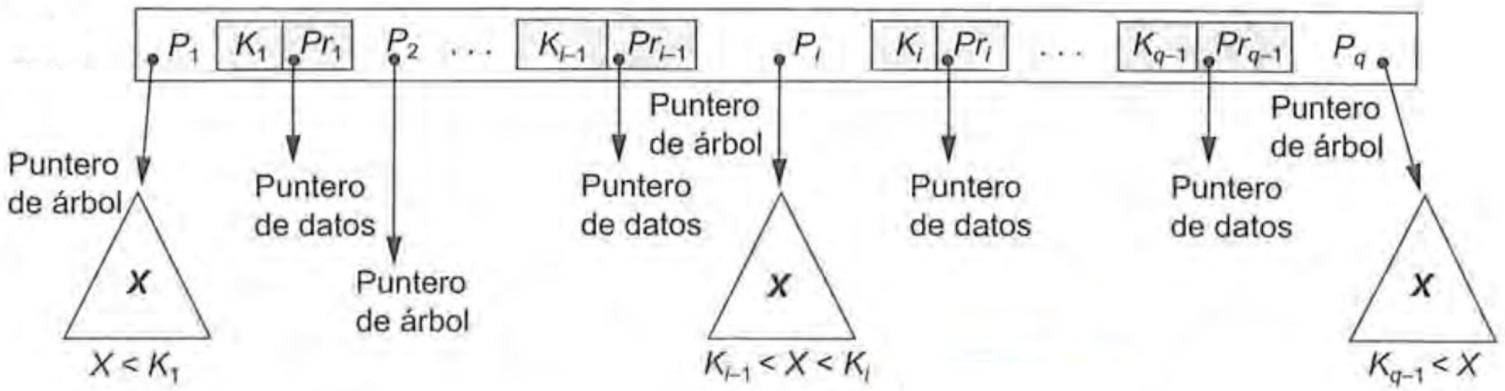


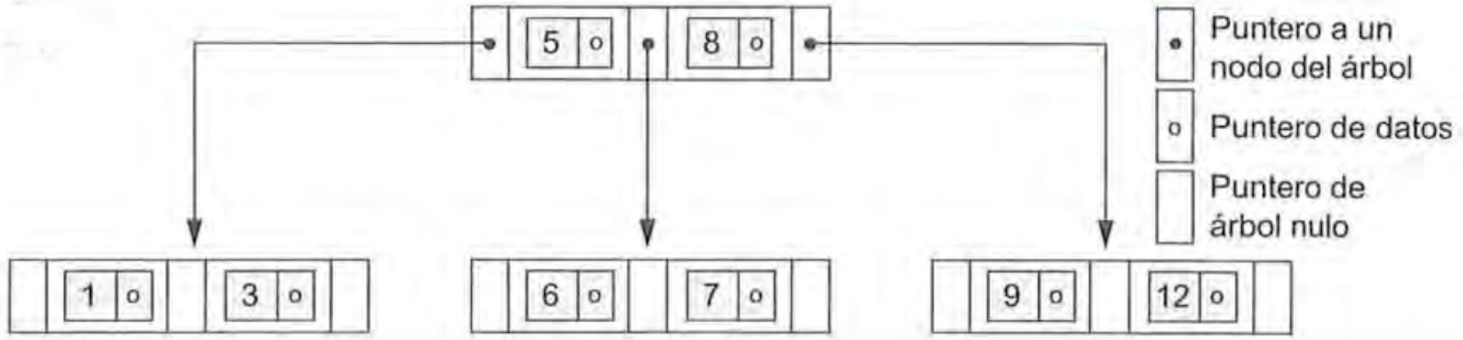
Árboles B

Un árbol B siempre estará equilibrado, lo cual evita desperdiciar espacio producto de borrados.

Cada nodo de un árbol B contiene punteros a datos y punteros a sub-árboles.

Cada nodo, excepto los nodos raíz y hoja, tienen por los menos (p/2) punteros de árbol





FACULTAD DE INGENIERÍA
UNIVERSIDAD DE TALCA

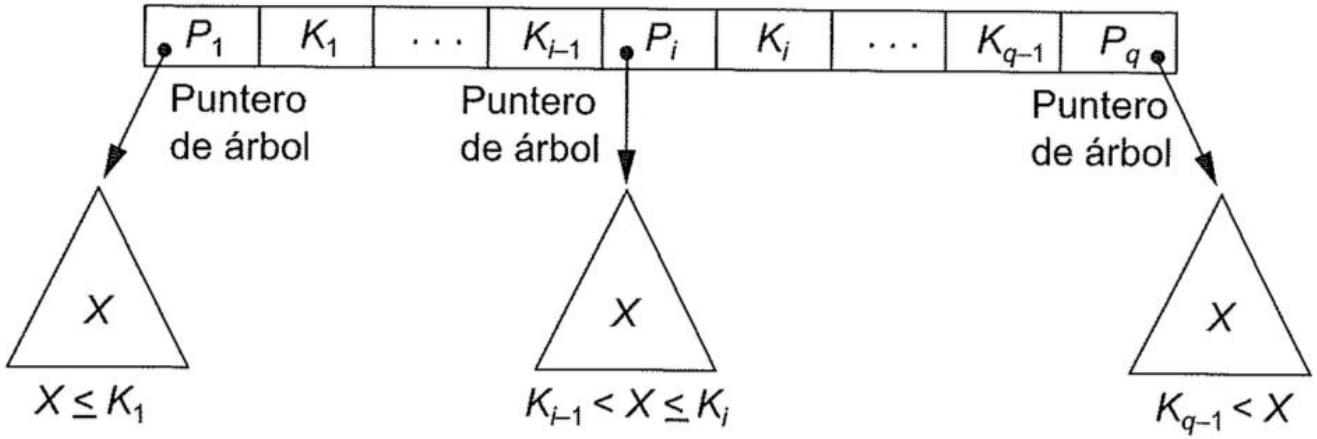
Árboles B+

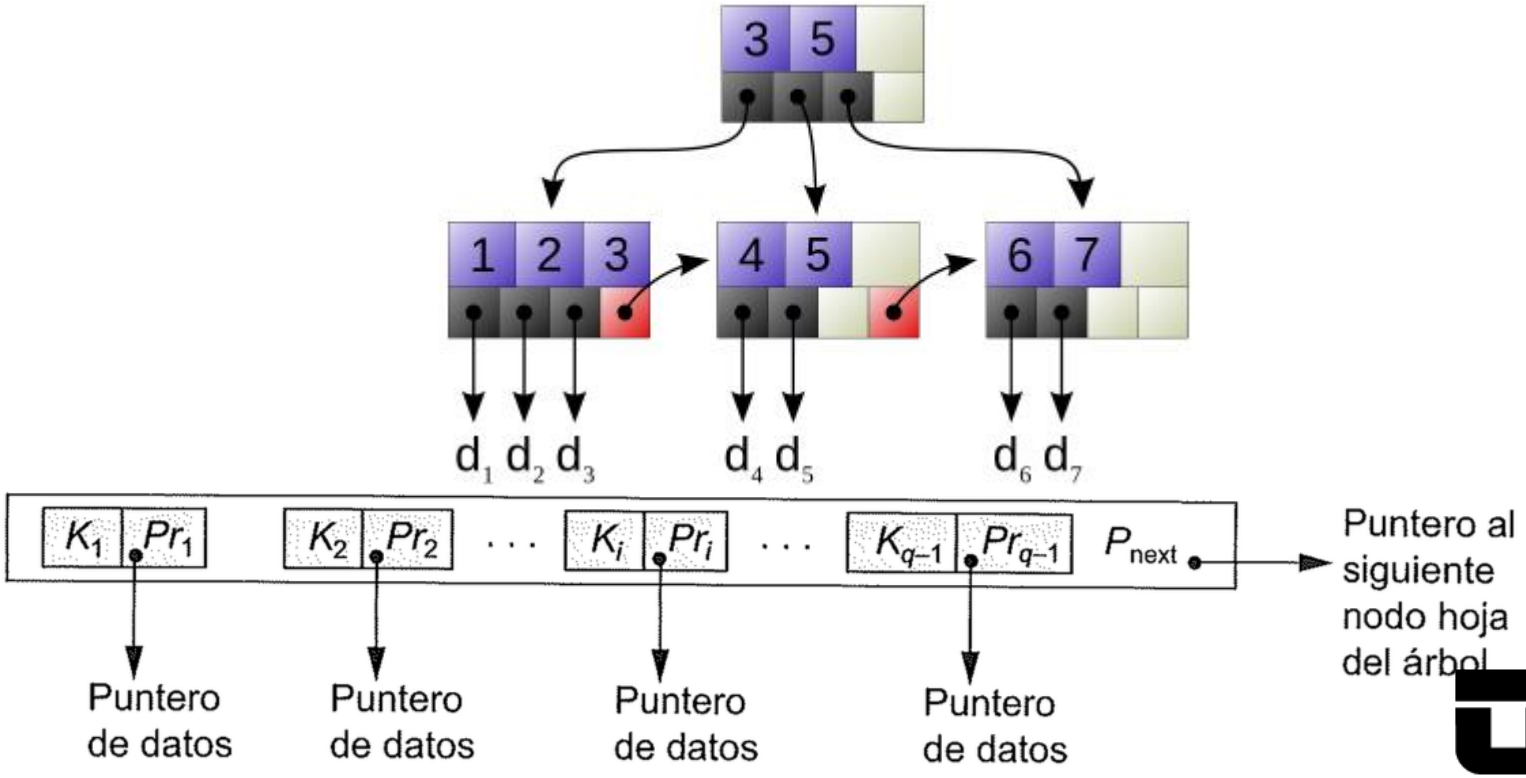
En un árbol B+, cada valor del campo de búsqueda aparece una vez en algún nivel del árbol.

Los punteros a datos solo se almacenan en los nodos hoja.

Los nodos internos solo contienen valores de búsqueda y punteros a los sub-árboles.

Los nodos hoja están enlazados para ofrecer un acceso ordenado.

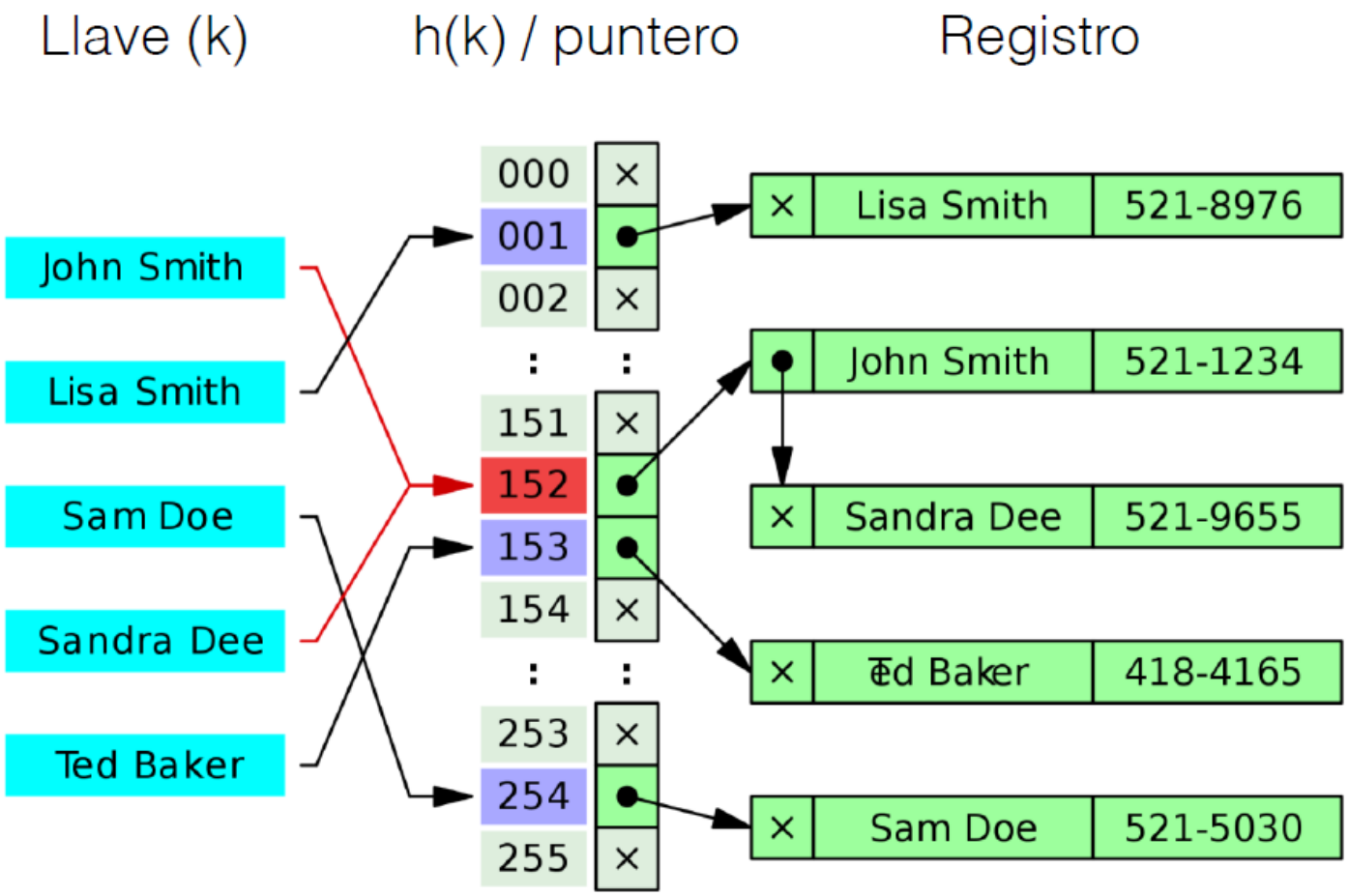




INGENIERÍA CIVIL EN COMPUTACIÓN

TALCA UNIVERSIDAD CHILE

Índice Hash


Llave (k)
h(k) / puntero
Registro
John Smith
Lisa Smith
Sam Doe
Sandra Dee
Ted Baker
000
001
002
151
152
153
154
253
254
255
Lisa Smith
521-8976
John Smith
521-1234
Sandra Dee
521-9655
Ted Baker
418-4165
Sam Doe
521-5030

Sobre el uso de índices

- ¿Cuándo indexar un atributo?
  - El atributo es clave
  - El atributo es usado en condiciones de selección
  - El atributo es usado en operaciones de JOIN
- ¿Cuándo usar un índice B+?
  - Para consultas de rango (18 < edad < 25)
- ¿Cuándo usar un índice Hash?
  - Para consultas de igualdad o para atributos usados en ***JOINs***

Directrices Básicas

- No indexar relaciones pequeñas
- Añadir un índice a cada clave principal
- Añadir un índice secundario a las claves externas que se usan frecuentemente
- Añadir índices a atributos que participan en ***JOINs*** así como en operaciones que requieren orden
- No indexar atributos que se actualizan frecuentemente
- No indexar atributos que contiene cadenas largas

Referencias

- "Fundamentos de Sistemas de Bases de Datos". R. A. Elmasri and S. B. Navathe. 7ta edición, 2007. (Capítulos 13, 14 y 16)
- "Sistemas de bases de datos". T. M. Connolly and C. E. Begg. 4ta edición, 2005. (Apéndice C)

Actividad – Laboratorio 1 – Parte 1

Comprender el propósito, funcionamiento y beneficios de los índices en bases de datos, analizando su impacto en el rendimiento de consultas de manera conceptual.

Imagina que eres parte de un equipo que administra una base de datos de una tienda online con la siguiente tabla:

Tabla: Clientes

- id_cliente
- nombre
- apellido
- email
- ciudad
- fecha_registro

La tabla tiene **1.000.000 de registros**.

Responde las siguientes preguntas:

- ¿Cómo se realizaría la búsqueda de un cliente por email sin usar índices?
- ¿Qué problema surge al buscar datos en tablas muy grandes?
- ¿Qué campos crees que se consultan con mayor frecuencia?

Actividad – Laboratorio 1 – Parte 2

A partir de las siguientes consultas frecuentes:

- ✓ Buscar cliente por ***email***
- ✓ Buscar clientes por ***ciudad***
- ✓ Ordenar clientes por ***fecha_registro***
- ✓ Buscar clientes por ***apellido***

Responder las siguientes preguntas:

- ¿En qué campos crearías índices? Justifica el ***motivo*** y el ***tipo de índice*** a utilizar.
- ¿Crearías índices en todos los campos? ¿Por qué?
- ¿Qué desventajas puede tener usar demasiados índices?

En parejas, realizar un breve reporte que permita dar respuesta a las preguntas planteadas.

Subirlo a Educandus al término de la jornada.

Pauta laboratorio 1 - Parte 1

1 ¿Cómo se realizaría la búsqueda de un cliente por email sin usar índices?

- Leer registro por registro (los 1.000.000)
- Comparar el campo email en cada fila
- Detenerse solo cuando encuentre coincidencias (o termine toda la tabla)

Esto es una búsqueda **O(n)** (lineal), donde *n = 1.000.000*

2 ¿Qué problema surge al buscar datos en tablas muy grandes?

El principal problema es el **rendimiento**:

- **Tiempo de respuesta alto**: mientras más crece la tabla, más lento se vuelve
- **Alto consumo de recursos**: CPU y disco trabajan más
- **Bloqueos o congestión**: múltiples consultas simultáneas empeoran el rendimiento

En resumen: Las consultas se vuelven lentas e ineficientes

Pauta laboratorio 1 - Parte 1

3 ¿Qué campos se consultan con mayor frecuencia?

En una tienda online, los campos más usados podrían ser:

Alta frecuencia

- **email** → login, recuperación de cuenta, validación
- **id_cliente** → clave primaria para búsquedas internas
- **fecha_registro** → reportes, análisis de crecimiento
- **ciudad** → segmentación, envíos, marketing

Frecuencia media

- **nombre / apellido** → búsquedas manuales o atención al cliente

Pauta laboratorio 1 - Parte 2

1 ¿En qué campos crear índices?

- **email** → índice Unique (B-Tree) // Se usa para búsquedas exactas.
- **ciudad** → índice no único (B-Tree) // Se usa en filtros.
- **fecha_registro** → índice no único (B-Tree) // Se usa en ordenamiento.
- **apellido** → índice no único (B-Tree) // útil para búsquedas específicas, riesgo de poca selectividad.

2 ¿Crearías índices en todos los campos?

- No. Crear índices en todos los campos **no es una buena práctica**.

3 ¿Qué desventajas tiene usar demasiados índices?

Tiene impactos en las transacciones de INSERT, UPDATE y DELETE, ya que se deben actualizar todos los índices. Inserciones más lentas y mayor costo en escritura. Si se aplican demasiado índices el motor de base de datos podría elegir uno que no sea el óptimo, podría empeorar el rendimiento.